

Network Traffic Analysis

By

K. Tharun Raj

CSE - Cyber Security

Bharat Institute of Engineering and Technology

May 2026

ABSTRACT

This project focuses on Network Traffic Analysis and Intrusion Detection to monitor and analyze network packets for identifying suspicious activities and security threats. The main purpose of the project is to understand how data travels through a network and how abnormal traffic can be detected using cybersecurity tools. Tools such as Snort, tcpdump, Wireshark, and Kali Linux were used to capture, monitor, and inspect network traffic. Different types of packets including TCP, UDP, SYN, and ACK packets were analyzed, and Snort rules were configured to detect suspicious traffic such as SYN scan attacks. The project successfully demonstrated packet capturing, traffic monitoring, and intrusion detection techniques, helping to improve understanding of network security and real-time traffic analysis.

INTRODUCTION

Network Traffic Analysis is the process of capturing, monitoring, and examining network packets to understand how data travels across a network. It helps in identifying normal and abnormal network behaviour by analysing different types of packets such as TCP, UDP, ICMP, SYN, and ACK packets. In cybersecurity, network traffic analysis is very important because it helps detect unauthorized access, suspicious activities, malware communication, and network-based attacks. Organizations continuously monitor network traffic to protect sensitive data, maintain network performance, and prevent cyber threats from affecting their systems.

This project focuses on packet analysis, intrusion detection, monitoring suspicious traffic, and detecting attacks using tools such as Snort, tcpdump, and Wireshark. The project involves capturing live network packets, analysing packet flow, and configuring intrusion detection rules to identify suspicious traffic such as SYN scan attacks. The main objective of this project is to understand network communication, analyse traffic patterns, and improve knowledge of real-time network monitoring and security analysis techniques used in cybersecurity environments.

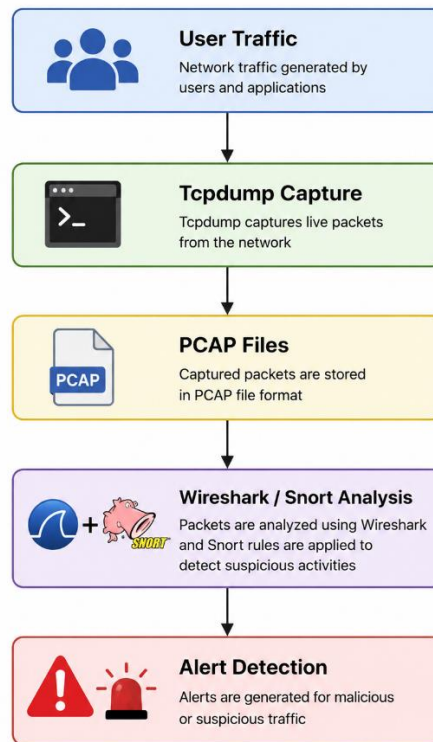
OBJECTIVES

- To capture live network packets from different network interfaces.
- To analyze TCP/IP traffic and understand packet communication.
- To monitor network activities in real time.
- To detect intrusion attempts using Snort
- To improve practical knowledge of cybersecurity and network monitoring techniques.

TOOLS AND TECHNOLOGIES

Tool	Purpose	Description
Kali Linux	Penetration Testing OS	Kali Linux was used as the main operating system for performing network monitoring and cybersecurity testing.
Snort	Intrusion Detection	Snort was used to detect suspicious network activities and generate alerts for possible attacks.
tcpdump	Packet Capturing	Tcpdump was used to capture live network packets and store them for analysis.
Wireshark	Packet Analysis	Wireshark was used to inspect and analyze packet details such as source IP, destination IP, protocols, and traffic flow.
Python	Data Processing	Python was used for processing captured data and automating analysis tasks.
Nmap	Port Scanning	Nmap was used to generate scanning traffic and test intrusion detection rules.

SYSTEMARCHITECTURE



The above workflow diagram explains the complete process of Network Traffic Analysis in a simple and step-by-step manner. First, user traffic is generated when users browse websites, send data, or use network applications. This traffic contains different types of network packets that travel through the network. Next, tcpdump is used to capture the live network packets from the system. These captured packets are then stored in PCAP (Packet Capture) files, which contain detailed information about network communication.

After capturing the packets, the PCAP files are analysed using Wireshark and Snort. Wireshark helps in viewing and understanding packet details such as IP addresses, protocols, and packet flow, while Snort applies intrusion detection rules to identify suspicious activities or possible attacks. Finally, if any malicious or abnormal traffic is detected, alert messages are generated. This workflow helps in monitoring network activity, detecting threats, and improving network security through real-time traffic analysis.

METHODOLOGY

Step 1: Environment Setup

- Installed Kali Linux for network traffic analysis and security testing.
- Installed required tools like Wireshark, nmap, snort and tcpdump.
- Configured Snort for intrusion detection and alert generation.
- Verified network interfaces such as eth0 and any before packet capturing.

This step prepared the environment required for capturing, monitoring, and analyzing network traffic.

Step 2: Packet Capturing

Command Used

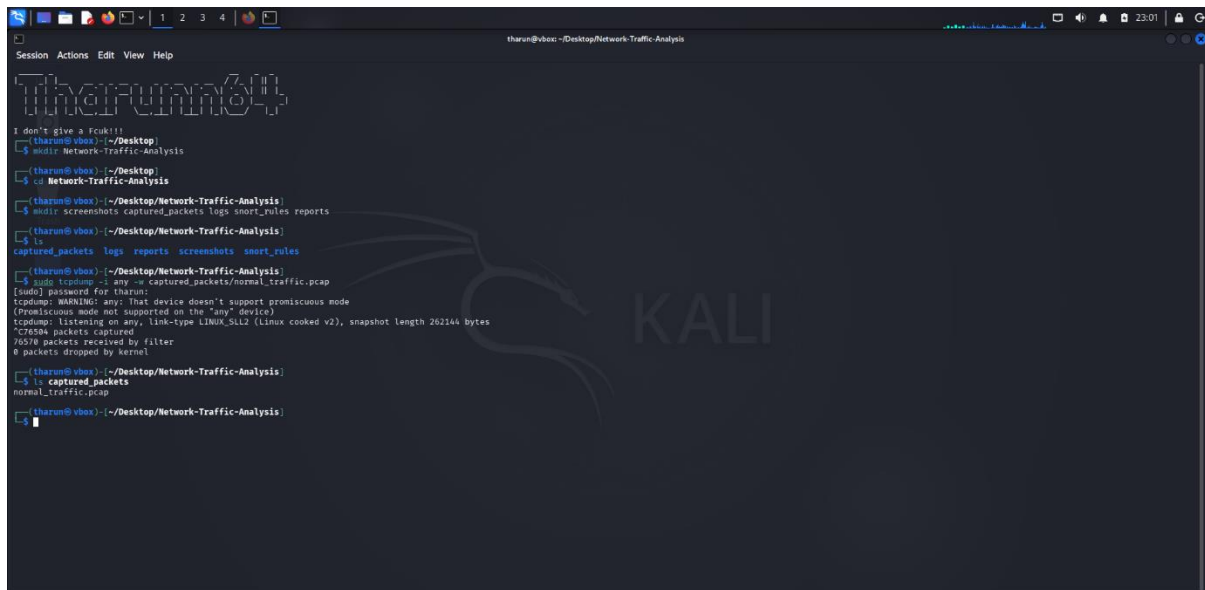
```
sudo tcpdump -i any -w normal_traffic.pcap
```

Explanation of Command

- **sudo** → Runs the command with administrator permission.
- **tcpdump** → Tool used to capture network packets.
- **-i any** → Captures packets from all network interfaces.
- **-w** → Saves captured packets into a file.
- **normal_traffic.pcap** → Name of the output file storing captured traffic.

Purpose

- Captured live network traffic.
- Stored captured packets into PCAP files for analysis.
- Captured both normal and suspicious traffic.



```
tharun@vbox: ~/Desktop/Network-Traffic-Analysis
Session Actions Edit View Help
I don't give a fck!!!
[tharun@vbox:~/Desktop]
$ mkdir Network-Traffic-Analysis
[tharun@vbox:~/Desktop]
$ cd Network-Traffic-Analysis
[tharun@vbox:~/Desktop/Network-Traffic-Analysis]
$ mkdir screenshots captured_packets logs snort_rules reports
[tharun@vbox:~/Desktop/Network-Traffic-Analysis]
$ ls
captured_packets  logs  reports  screenshots  snort_rules
[tharun@vbox:~/Desktop/Network-Traffic-Analysis]
$ sudo tcpdump -i any -w captured_packets/normal_traffic.pcap
[sudo] password for tharun:
tcpdump: WARNING: any: That device doesn't support promiscuous mode
(Promiscuous mode not supported on the "any" device)
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
^C76504 packets captured
76574 packets received by filter
0 packets dropped by kernel
[tharun@vbox:~/Desktop/Network-Traffic-Analysis]
$ ls captured_packets
normal_traffic.pcap
[tharun@vbox:~/Desktop/Network-Traffic-Analysis]
```

This image shows the setup of a Network Traffic Analysis project in Kali Linux. The user created organized folders for storing captured packets, logs, reports, and Snort rules. Using the tcpdump command, live network traffic was captured and saved into a .pcap file for later cybersecurity analysis.

Step 3: Traffic Generation

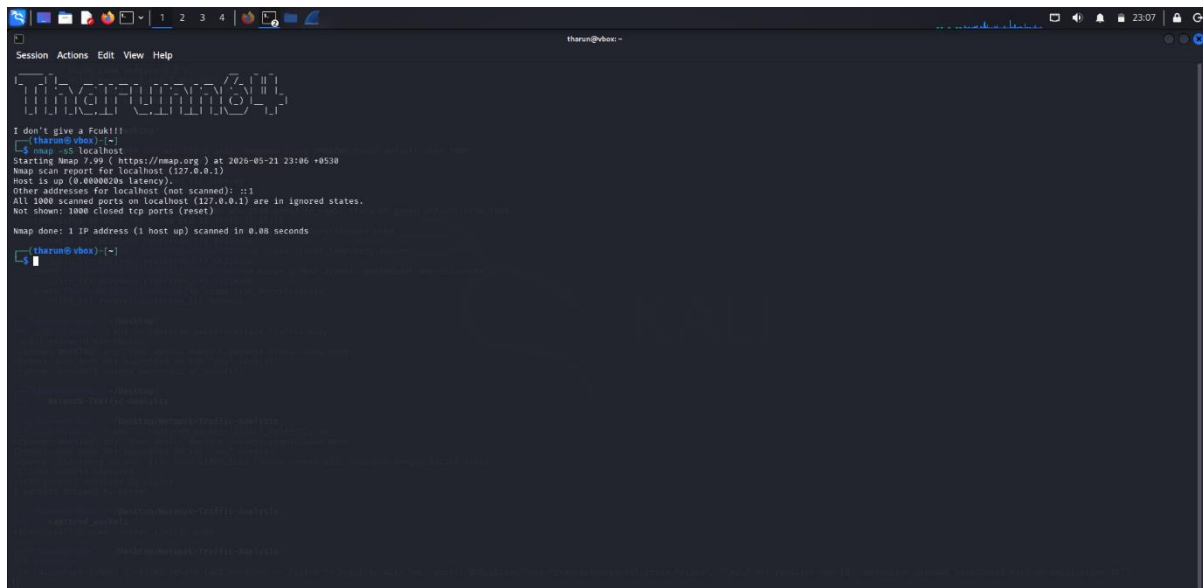
Command Used

nmap -sS localhost

Traffic Generated

- Ping traffic using ICMP packets.
- Web browsing traffic using HTTP requests.
- DNS request traffic.
- TCP SYN scan traffic using Nmap.

Different types of traffic were generated to study packet behaviour and test intrusion detection rules.



```
tharun@kali:~$ nmap -sS localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2020-05-21 23:40 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000000s latency).
Other addresses for localhost (not scanned): ::1
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)
Nmap done: 1 IP address (1 host up) scanned in 0.08 seconds
```

This image shows an Nmap SYN scan being performed on the localhost system in Kali Linux. The scan checks for open ports and network services running on the machine. The output indicates that the host is active and demonstrates basic traffic generation for network traffic analysis and Snort detection testing.

Step 4: Snort Rule Configuration

Custom Rule

A custom Snort rule is used to detect specific types of suspicious network traffic. In this project, a rule was created to identify TCP SYN scan attacks generated using Nmap. The rule continuously monitors network packets and generates an alert whenever suspicious SYN packets are detected.

```
alert tcp any any -> any any (msg:"SYN Scan Detected"; flags:S;
sid:1000001;)
```

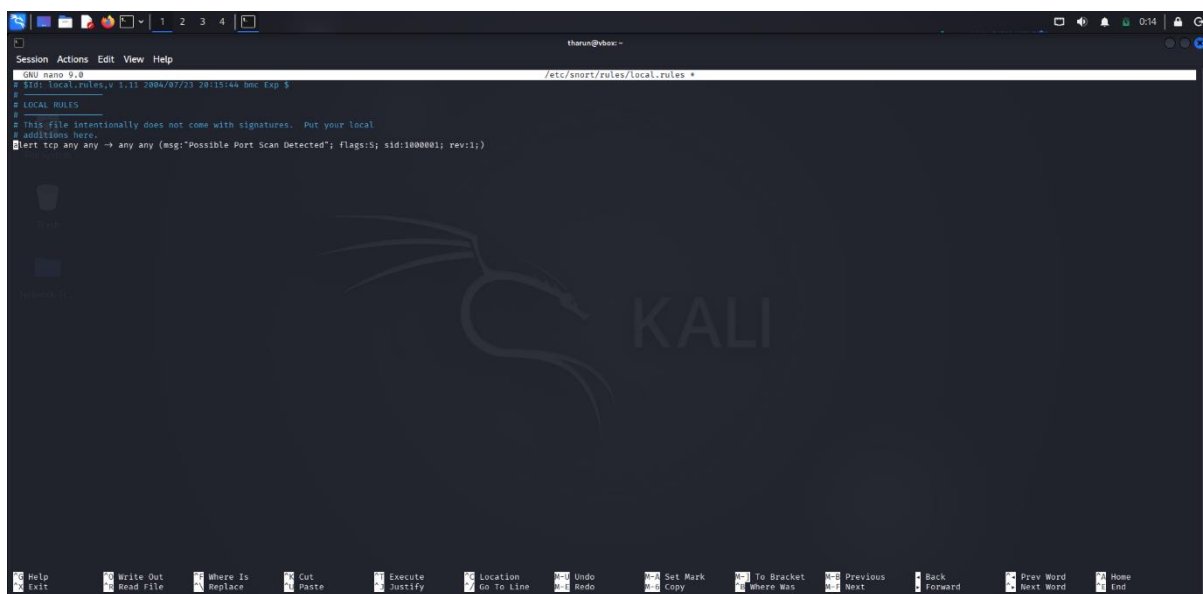
Explanation of Rule

- **alert** → Generates an alert when suspicious traffic is detected.
- **tcp** → Monitors TCP packets.

- **msg** → Displays alert message.
- **flags:S** → Detects TCP SYN packets.
- **sid** → Unique ID for the Snort rule.

Purpose

- Configured Snort custom rules.
- Detected suspicious SYN scan traffic.
- Generated alerts for possible port scanning attacks.



This image shows a custom Snort rule being created in the `local.rules` file to detect possible port scan activity. The rule monitors TCP SYN packets and generates alerts when suspicious scanning behavior is detected.

Step 5: Packet Analysis

Filters Used

tcp.flags.syn == 1

tcp.flags.ack == 1

Analysis Performed

- Analyzed TCP packets for network communication.
- Examined SYN packets used to start TCP connections.
- Analyzed ACK packets used for acknowledgment in communication.
- Studied TCP three-way handshake process (SYN → SYN-ACK → ACK).
- Monitored DNS, HTTP, and ICMP traffic using Wireshark filters.

Packet analysis helped in understanding network communication and identifying suspicious traffic patterns using protocol filters.

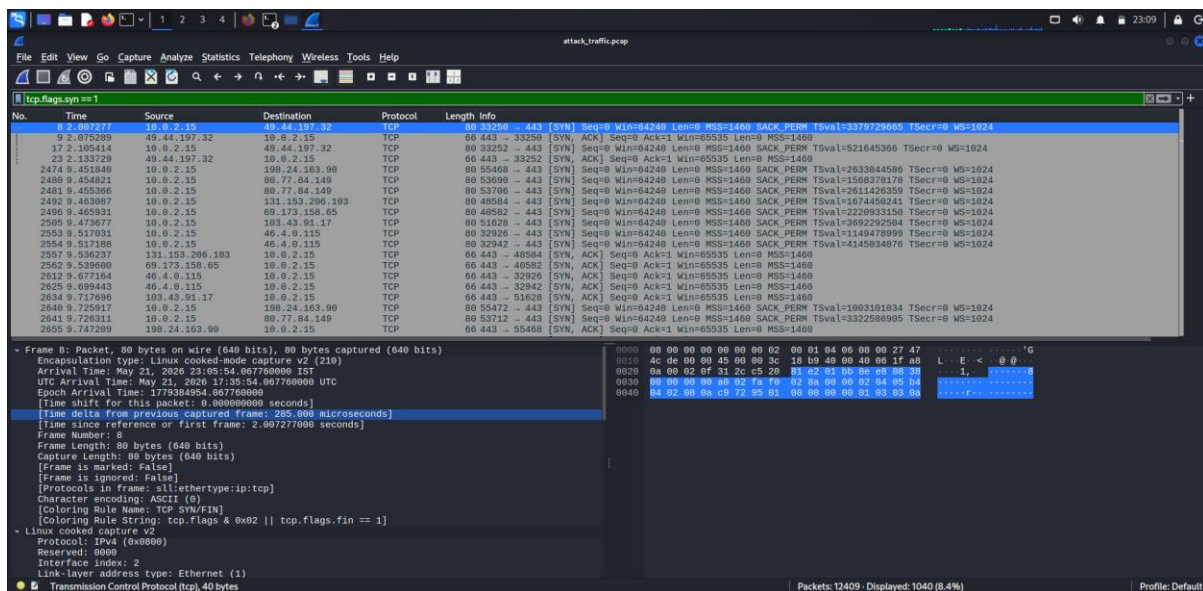


Figure: TCP SYN Packet Analysis

This figure shows TCP SYN packets filtered using `tcp.flags.syn == 1` in Wireshark. SYN packets are used to initiate TCP connections during the TCP three-way handshake process. Multiple SYN packets can indicate connection attempts or network scanning activities such as SYN scans. This analysis is useful for detecting suspicious traffic and understanding TCP connection establishment.

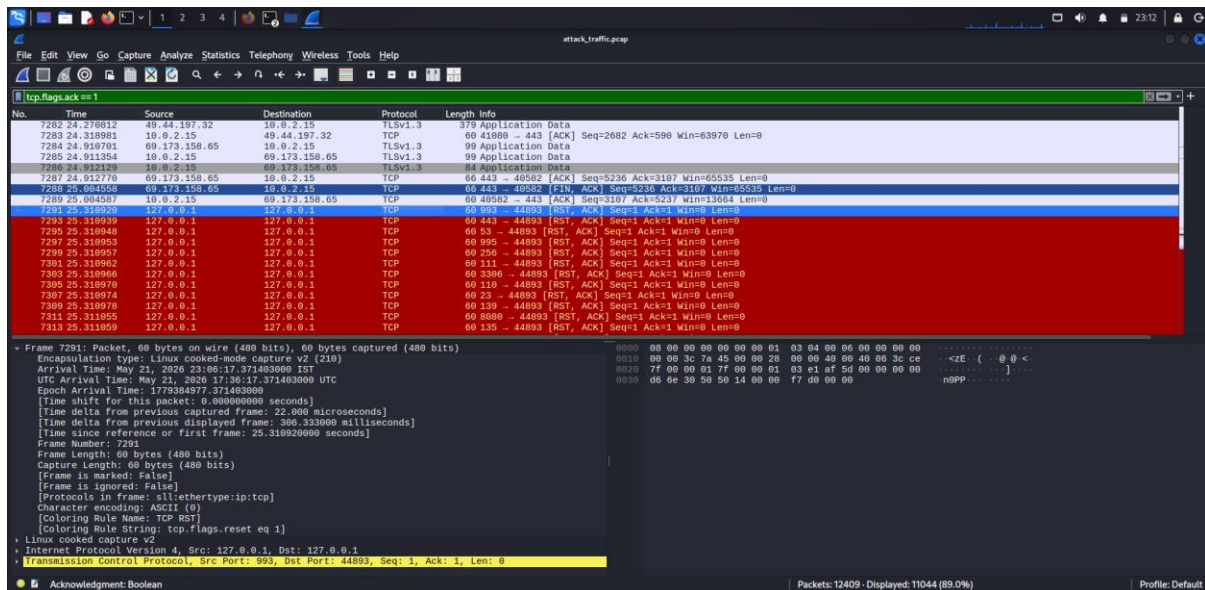


Figure : TCP ACK Packet Analysis

This figure shows TCP packets with the ACK flag enabled, filtered using `tcp.flags.ack == 1`. ACK packets are used in TCP communication to acknowledge successful receipt of data packets between systems. The capture demonstrates active TCP sessions and packet acknowledgments during network communication. This analysis helps understand reliable data transfer mechanisms in TCP connections.

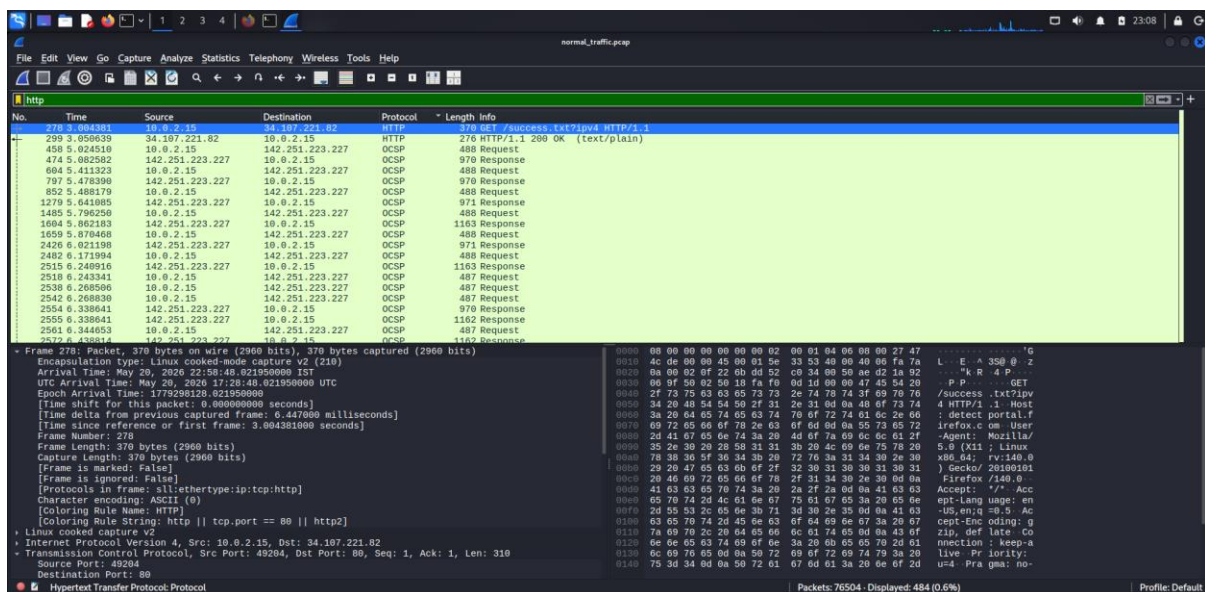


Figure : HTTP Protocol Traffic Analysis

This figure shows HTTP packets captured and analyzed using Wireshark. The HTTP filter was applied to display web communication traffic between source and destination systems. Packet details such as GET requests, destination IP address, and protocol information can be observed. This analysis helps understand normal web browsing traffic and application-layer communication in the network.

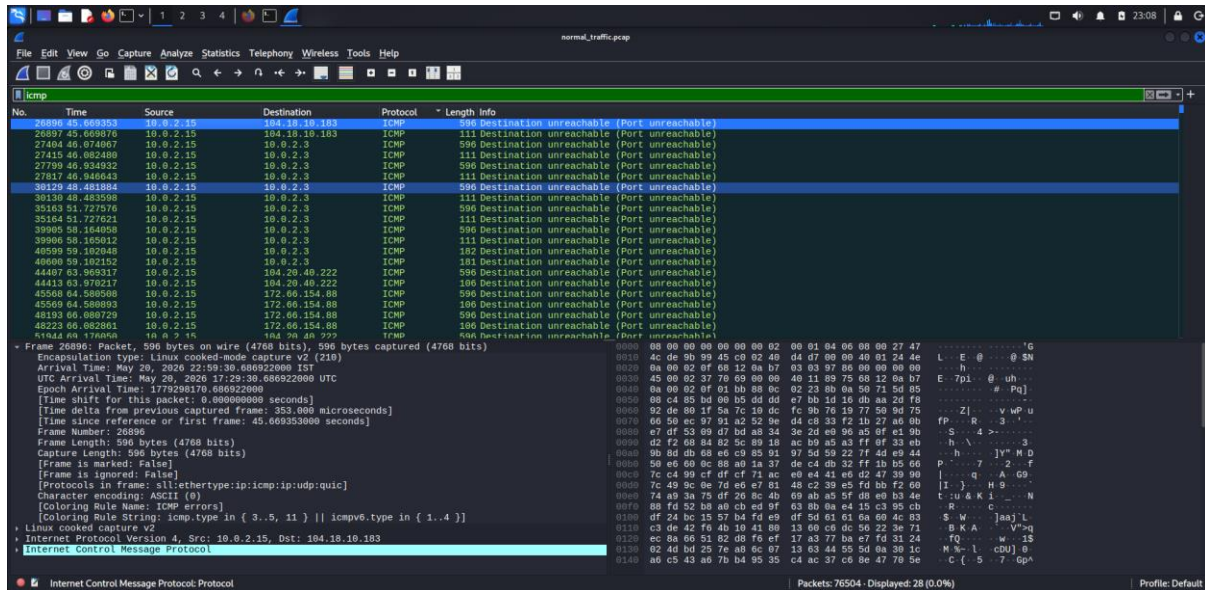
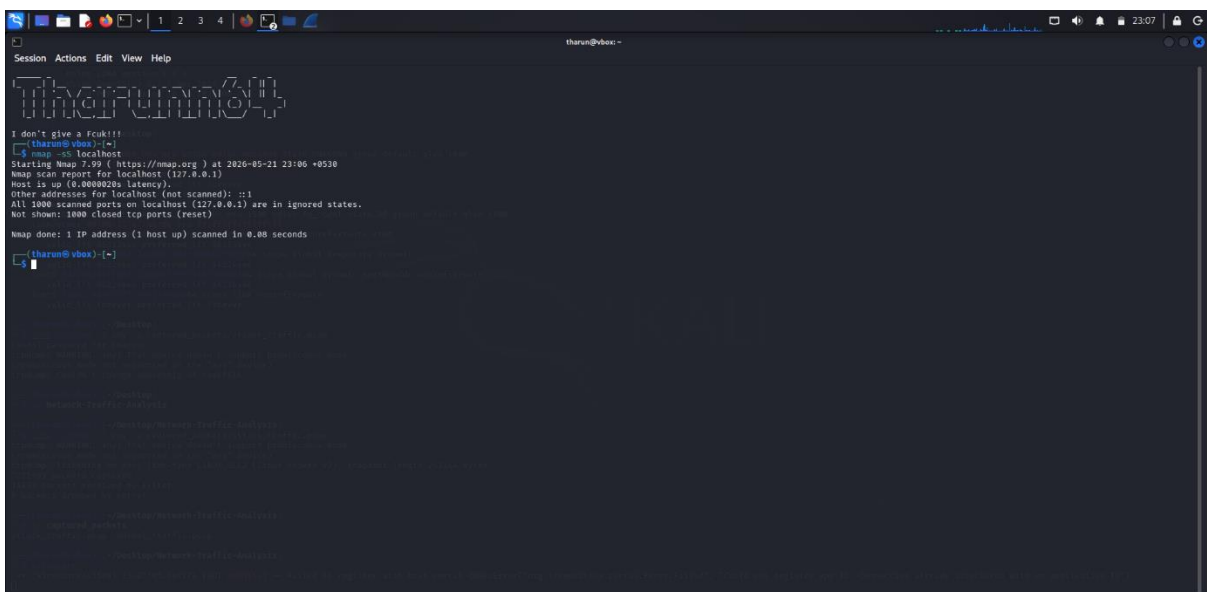


Figure 2: ICMP Protocol Traffic Analysis

This figure displays ICMP packets captured in Wireshark using the ICMP filter. The packets mainly show “Destination Unreachable” messages exchanged between systems. ICMP is commonly used for network diagnostics and error reporting purposes. Analyzing ICMP traffic helps identify connectivity issues and network communication problems.

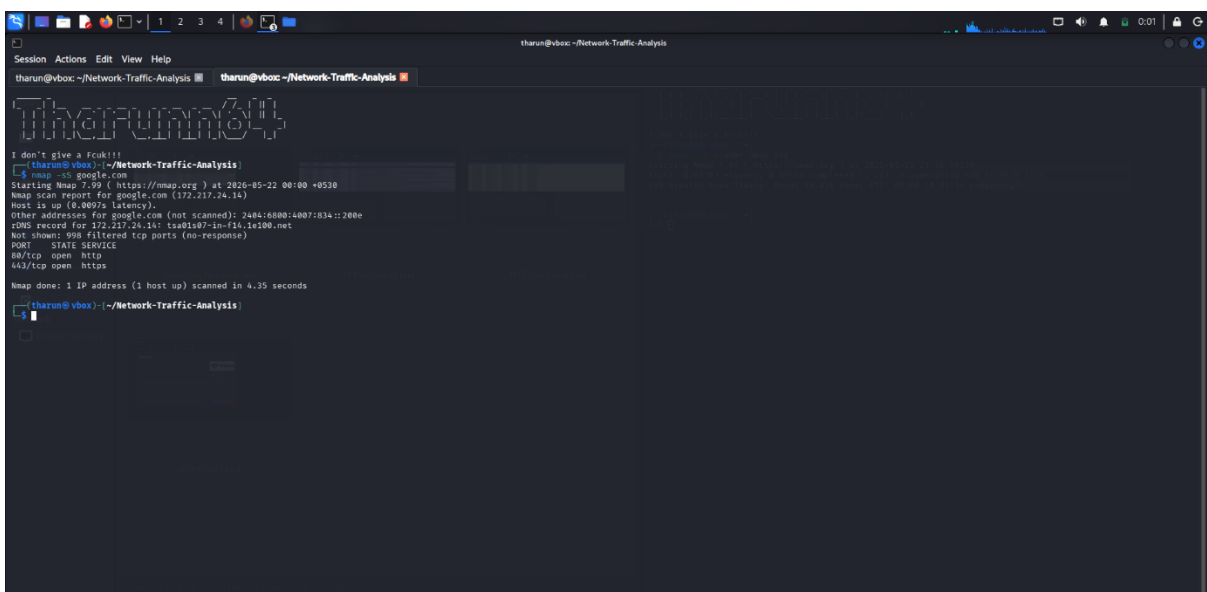
RESULTS and OBSERVATIONS

The project successfully demonstrated network traffic capture, traffic analysis, and intrusion detection using tools such as Tcpdump, Wireshark, Nmap, and Snort. Normal and suspicious network traffic was generated and stored in .pcap files for analysis. Wireshark was used to inspect protocols such as HTTP, DNS, ICMP, TCP SYN, and TCP ACK packets, helping to understand both normal communication and attack-related traffic patterns.



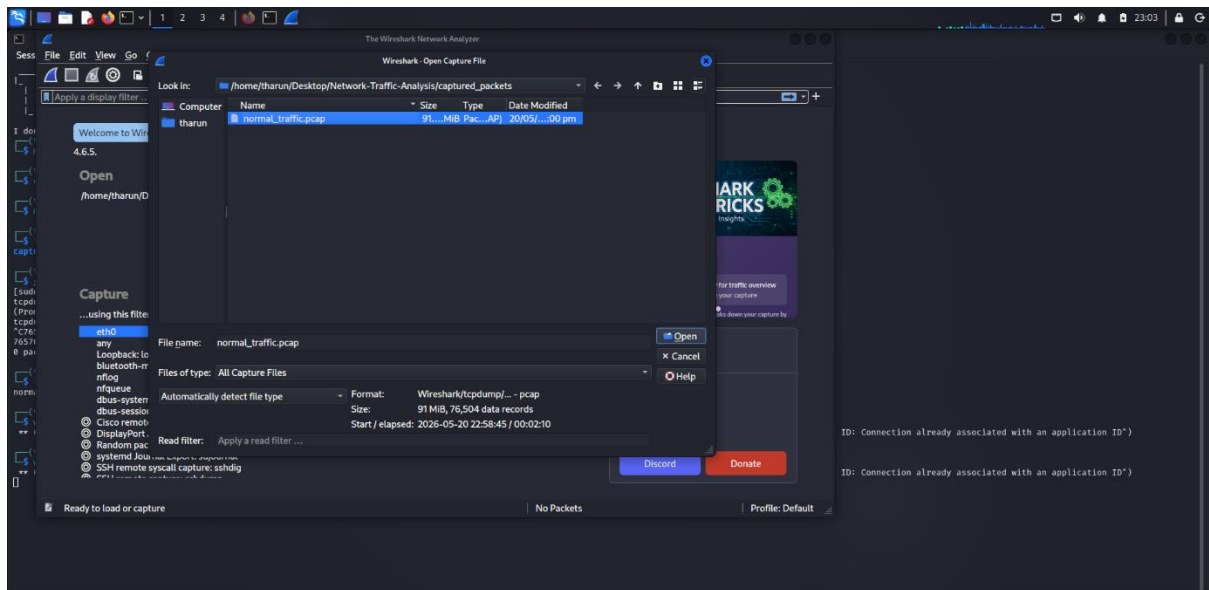
```
tharun@vbox:~$ nmap -sS localhost
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-21 23:00 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000002s latency).
Other addresses for localhost (not scanned): ::1
All 2000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)
Nmap done: 1 IP address (1 host up) scanned in 0.08 seconds
```

Nmap SYN scan

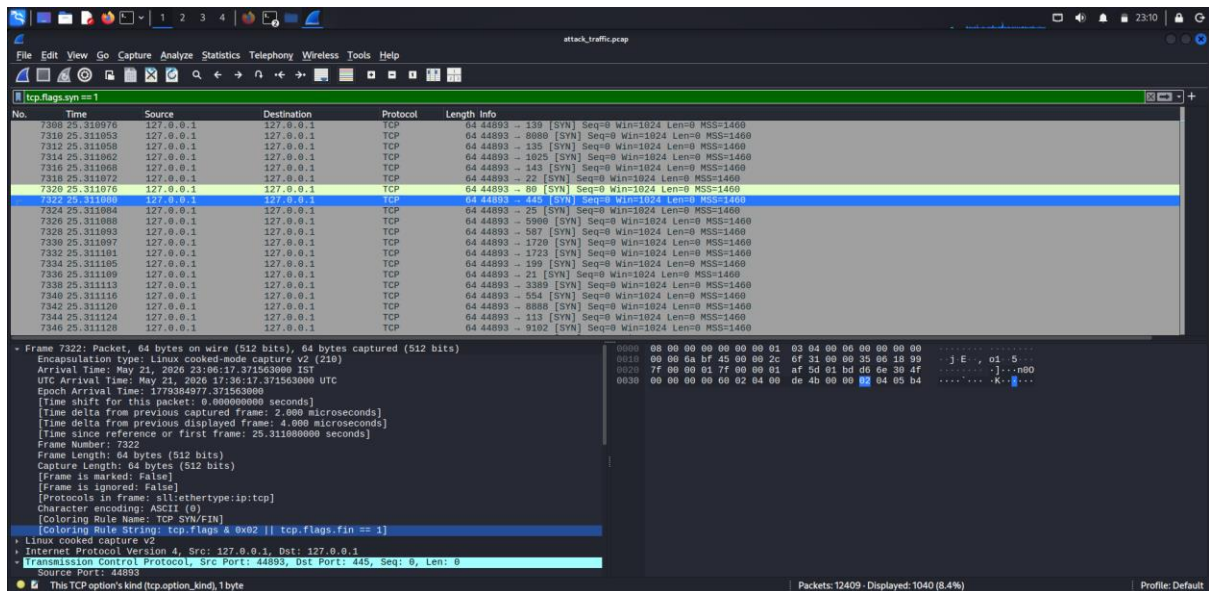


```
tharun@vbox:~/Network-Traffic-Analysis$ nmap -sS google.com
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 00:00 +0530
Nmap scan report for google.com (172.217.24.14)
Host is up (0.0007s latency).
Other addresses for google.com (not scanned): 2404:6800:4007:834::200e
DNS record for 172.217.24.14: tsal0107-in-f14.1e100.net
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
443/tcp   open  https
Nmap done: 1 IP address (1 host up) scanned in 4.35 seconds
```

Creating Attack traffic using nmap



Opening captured traffic .pcap file



Analysing traffic using wireshark

```
tharun@vbox:~$ nano /etc/snort/rules/local.rules
# This file intentionally does not come with signatures. Put your local
# additions here.
#
# alert tcp any any -> any any (msg:'Possible Port Scan Detected'; flags:S; sid:1000001; rev:1)

tharun@vbox:~$
```

Configuring snort rules for alerts

```
tharun@vbox:~/Network-Traffic-Analysis$ nano /etc/snort/snort.lua
# This file intentionally does not come with signatures. Put your local
# additions here.
#
# alert tcp any any -> any any (Flags:S; msg:'Possible Port Scan Detected'; sid:1000001;)
# alert tcp any any -> any any (msg:'Possible Port Scan Detected'; flags:S; sid:1000001; rev:1)

tharun@vbox:~/Network-Traffic-Analysis$ nano /etc/snort/snort.lua
tharun@vbox:~/Network-Traffic-Analysis$ nano /etc/snort/rules/local.rules
tharun@vbox:~/Network-Traffic-Analysis$ snort -A alert-fast -l /etc/snort/snort.lua -i eth0
snort - Snort++ 3.12.2.0

Loading /etc/snort/snort.lua:
ERROR: /etc/snort/snort.lua: can't load /etc/snort/snort.lua: /etc/snort/snort.lua:19: '}' expected (to close '[' at line 189) near 'variables'

pcap DAQ configured to passive.
FATAL: See prior 1 errors (0 warnings)
Fatal Error, Quitting..

tharun@vbox:~/Network-Traffic-Analysis$ nano /etc/snort/snort.lua
tharun@vbox:~/Network-Traffic-Analysis$ nano /etc/snort/rules/local.rules
tharun@vbox:~/Network-Traffic-Analysis$ snort -A alert-fast -l /etc/snort/rules/local.rules -i eth0
snort - Snort++ 3.12.2.0

Loading rule args:
loading /etc/snort/rules/local.rules:
Finished /etc/snort/rules/local.rules:
Finished rule args:

Rule counts
total rules loaded: 1
text rules: 1
option chains: 1
chain headers: 1

port rule counts
      tcp  udp  icmp  ip
any      1    0    0    0
total    1    0    0    0

pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
** [0] eth0
```

Starting snort service


```
Session Actions Edit View Help
tuning
runtime: 00:03:12
seconds: 192.745961
bits/sec: 73
Mbps/sec: 1
o)- Snort exiting

(tharun@vbox: ~) ~/Network-Traffic-Analysis
$ sudo snort -c /etc/snort/rules/local.rules -i eth0 -A fast
o)- Snort++ 3.12.2.0

Loading rule args:
Loading /etc/snort/rules/local.rules:
Finished /etc/snort/rules/local.rules:
Finished rule args:

rule counts
total rules loaded: 1
text rules: 1
option chains: 1
chain headers: 1

port rule counts
tcp      udp      icmp     ip
any      1        0        0        0
total    1        0        0        0

pcap DAG configured to passive.
Commencing packet processing.
Retry queue interval is: 200 ms
** [!] eth0
05/21-23:58:05.700138 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36133 -> 45.33.32.15
05/21-23:58:05.118999 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.15
05/21-23:58:05.118994 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.15
05/21-23:58:05.118148 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.15
05/21-23:58:05.118366 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.15
05/21-23:58:05.118487 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.15
05/21-23:58:05.118536 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.15
05/21-23:58:05.118599 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.15
05/21-23:58:05.118648 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.15
05/21-23:58:05.118694 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.15
05/21-23:58:05.118725 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.15
05/21-23:58:07.218862 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.15
05/21-23:58:07.218866 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.15
05/21-23:58:07.218867 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.15
05/21-23:58:07.219257 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.15
05/21-23:58:07.219342 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.15
05/21-23:58:07.219367 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.15
05/21-23:58:07.219374 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.15
05/21-23:58:07.219772 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.15
05/21-23:58:07.219822 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.15
```

Testing Alert detection using nmap

```
Session Actions Edit View Help
tharun@vbox: ~) ~/Network-Traffic-Analysis
o)- Snort exiting

(tharun@vbox: ~) ~/Network-Traffic-Analysis
$ sudo snort -c /etc/snort/rules/local.rules -i eth0 -A fast
o)- Snort++ 3.12.2.0

Loading rule args:
Loading /etc/snort/rules/local.rules:
Finished /etc/snort/rules/local.rules:
Finished rule args:

rule counts
total rules loaded: 1
text rules: 1
option chains: 1
chain headers: 1

port rule counts
tcp      udp      icmp     ip
any      1        0        0        0
total    1        0        0        0

pcap DAG configured to passive.
Commencing packet processing.
Retry queue interval is: 200 ms
** [!] eth0
05/21-23:58:05.700138 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36133 -> 45.33.32.156:443
05/21-23:58:05.118999 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.156:111
05/21-23:58:05.118994 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.156:587
05/21-23:58:05.118148 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.156:199
05/21-23:58:05.118366 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.156:135
05/21-23:58:05.118487 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.156:2389
05/21-23:58:05.118536 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.156:445
05/21-23:58:05.118599 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.156:8080
05/21-23:58:05.118648 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.156:73
05/21-23:58:05.118694 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.156:113
05/21-23:58:05.118725 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.156:139
05/21-23:58:07.218862 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.156:139
05/21-23:58:07.218866 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.156:113
05/21-23:58:07.218867 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.156:23
05/21-23:58:07.219257 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.156:8080
05/21-23:58:07.219342 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.156:445
05/21-23:58:07.219367 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.156:135
05/21-23:58:07.219374 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.156:199
05/21-23:58:07.219772 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.156:107
05/21-23:58:07.219822 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36391 -> 45.33.32.156:111
05/21-23:58:07.219880 [*] [1:1000001:1] "Possible Port Scan Detected" ** [Priority: 0] [TCP] 10.0.2.15:36389 -> 45.33.32.156:22
```

Successfully alert are detected in terminal

Snort intrusion detection rules were successfully configured to detect TCP SYN scan activity generated using Nmap. During the attack simulation, Snort generated multiple alerts indicating possible port scanning attempts. The alerts included source IP addresses, destination IP addresses, protocols, and targeted ports, confirming successful real-time detection of suspicious traffic.

The observations from the packet analysis showed that TCP SYN packets were heavily used during scanning activity, while ACK packets represented established network communication. DNS and HTTP packets represented normal browsing

activity, whereas repeated SYN requests indicated possible reconnaissance or scanning behavior. The captured results demonstrate how network monitoring and intrusion detection systems can be used to identify malicious traffic and improve network security monitoring.

Overall, the experiment successfully achieved the objectives of capturing network traffic, analyzing packets using Wireshark, generating attack traffic using Nmap, and detecting suspicious activities using Snort IDS.

CHALLENGES FACED

During the project, several challenges were encountered while configuring and testing the network traffic analysis environment. Initial issues occurred during Snort configuration due to syntax errors in the rule files and incorrect configuration settings. These errors prevented Snort from starting properly until the rule syntax and configuration files were corrected.

- Snort configuration errors while creating custom detection rules
- Permission issues during packet capturing using Tcpdump
- Difficulty selecting the correct network interface for monitoring
- False positive alerts generated for normal TCP traffic
- Understanding and filtering different protocols in Wireshark

CONCLUSION

This project successfully demonstrated the process of capturing, monitoring, and analyzing network traffic using tools such as tcpdump, Wireshark, and Snort. Different types of network packets including TCP, UDP, SYN, and ACK packets were captured and examined to understand how communication takes place between devices in a network. Packet capture files were analyzed to study packet flow, source and destination IP addresses, protocols, and traffic behavior. Snort intrusion detection rules were configured to detect suspicious activities such as TCP SYN scan attacks generated during testing. The project successfully showed how network monitoring tools can be used to identify abnormal traffic and improve network security.

Through this project, valuable knowledge about network protocols, packet structures, intrusion detection, and real-time traffic monitoring was gained. The project helped in understanding how cybersecurity professionals analyze network traffic to identify suspicious activities and possible attacks before they affect the system. It also highlighted the importance of network traffic analysis in protecting organizations from cyber threats, maintaining network performance, and detecting unauthorized access attempts. Overall, the project provided practical experience with cybersecurity tools and improved understanding of how traffic analysis plays an important role in network security and attack detection.

FUTURE ENHANCEMENTS

In the future, this project can be improved by adding more advanced features for better network monitoring and security. A live dashboard can be created using Python to display network traffic, packet details, and alerts in real time. Machine learning techniques can also be added to automatically detect unusual or suspicious activities in the network. An automated alert system can help send warning messages whenever malicious traffic is detected. The project can also be expanded with cloud-based monitoring so that network traffic can be analyzed and monitored remotely from anywhere. These improvements can make the system faster, smarter, and more efficient for real-time cybersecurity monitoring.

REFERENCES

1. William Stallings, *Network Security Essentials: Applications and Standards*, Pearson Education.
2. Behrouz A. Forouzan, *Data Communications and Networking*, McGraw Hill Education.
3. Charles P. Pfleeger and Shari Lawrence Pfleeger, *Security in Computing*, Pearson Education.
4. [Snort Official Documentation](#)
5. [Tcpdump Manual Documentation](#)
6. [Wireshark Official Documentation](#)
7. [Kali Linux Documentation](#)
8. [Nmap Reference Guide](#)